

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验 |LAB5

OpenMP 并行矩阵乘法

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 吴迪、刘学正

起 始 日 期: 2026 年 5 月 13 日

结 束 日 期: 2026 年 5 月 15 日

实 验 地 点: B202

目录

1 实验简述 **2**

2 分析设计 **2**

2.1 OpenMP 线程模型 2
2.2 负载均衡与调度策略 2
2.3 访存局部性优化 3
2.4 并行化策略 3

3 核心代码 **4**

3.1 并行矩阵乘法实现 4
3.2 运行时调度设置 4
3.3 关键指令解析 4
3.4 变量作用域 4

4 结果分析 **5**

4.1 编译与运行环境 5
4.2 程序编译与运行验证 5
4.3 不同规模下的加速比 7
4.4 Static 调度性能数据 7
4.5 调度模式对比 8
4.6 三种调度模式在大规模矩阵上的对比 8
4.7 并行效率分析 9
4.8 线程扩展性 9
4.9 性能瓶颈分析 10

5 实验总结 **10**

5.1 技术收获 10
5.2 问题与反思 10

1　实验简述

本实验旨在实现基于 OpenMP 的并行矩阵乘法（GEMM, General Matrix Multiply），通过多线程并行化加速矩阵乘法运算，并系统分析不同调度策略（static、dynamic、guided）和线程数对性能的影响。

矩阵乘法是科学计算的核心操作之一，计算复杂度为 $O(n^3)$，在深度学习、数值模拟等领域有广泛应用。对于 $m \times n$ 矩阵 A 与 $n \times k$ 矩阵 B 的乘法 $C = A \times B$，其中每个元素满足：

$$C_{ij} = \sum_{p=0}^{n-1} A_{ip} \times B_{pj} \tag{1}$$

实验目标包括：

1. 掌握 OpenMP 的 `#pragma omp parallel for` 并行化方法
2. 理解不同调度策略对负载均衡的影响
3. 分析访存模式与 Cache 局部性对性能的优化作用
4. 评估并行加速比和并行效率

2　分析设计

2.1　OpenMP 线程模型

OpenMP 采用 Fork-Join 并行模型：

- **Fork**: 主线程遇到并行区域时，创建一组工作线程
- **Join**: 并行区域结束时，工作线程同步并退出，主线程继续执行

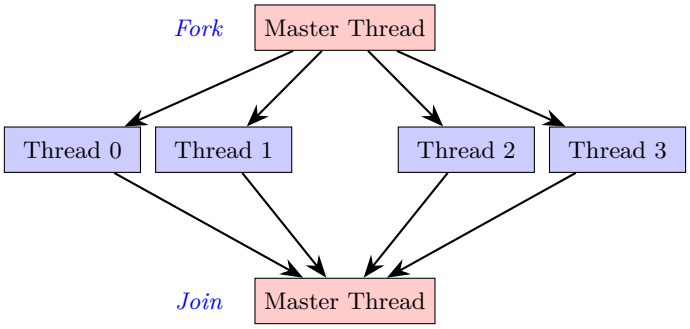


图 1: OpenMP Fork-Join 并行模型

在本实验中，使用 `#pragma omp parallel for` 将外层循环的任务分配给多个线程执行。各线程独立处理分配到的矩阵行，通过共享内存访问矩阵数据。

2.2　负载均衡与调度策略

OpenMP 提供三种调度模式：

调度模式	特点
`static`	编译时静态分配任务，均匀切分迭代空间。调度开销最小，适合各迭代工作量均衡的场景
`dynamic`	运行时动态分配，线程完成当前块后请求下一块。负载均衡好，但调度开销较大
`guided`	初始分配大块，逐渐减小块大小。折中方案，兼顾负载均衡和调度开销

表 1: OpenMP 调度模式对比

对于矩阵乘法，各迭代计算量相等，理论上 `static` 调度最优。但在实际运行中，NUMA 架构和访存竞争可能导致非理想行为。本实验通过 `schedule(runtime)` 配合 `omp_set_schedule()` 在运行时选择调度策略，避免硬编码。

2.3 访存局部性优化

Cache 局部性对矩阵乘法性能至关重要。三种循环顺序的性能差异如下：

- **ijk 顺序**: $A[i][p]$ 连续访问（行主序），$B[p][j]$ 跨步访问（步长为 k），Cache 命中率低
- **jik 顺序**: $B[p][j]$ 连续访问，但 $C[i][j]$ 频繁写入，无法复用累加器
- **ikj 顺序**: **本实验采用**。$A[i][p]$ 在 p 固定时广播到寄存器，在内层 j 循环中复用 n 次；$B[p][j]$ 和 $C[i][j]$ 均为连续访问，Cache 友好

循环顺序	A 访问模式	B 访问模式	C 访问模式
ijk	连续（行）	跨步（列）	连续（行）
jik	跨步（列）	连续（行）	连续（行）
ikj	**寄存器复用**	**连续（行）**	**连续（行）**

表 2: 不同循环顺序的访存模式对比

2.4 并行化策略

本实验采用以下并行化方案：

1. 使用 `schedule(runtime)` 配合 `omp_set_schedule()` 实现运行时调度选择，避免 `schedule` 子句不支持 C 变量的限制
2. 仅并行化外层 i 循环，因为 p 循环存在对 $C[i][j]$ 的累加依赖，若并行化 p 循环需额外同步
3. 循环内部使用局部变量 `a_ip` 缓存 $A[i][p]$，在内层 j 循环中复用，减少访存次数
4. 使用 `aligned_alloc(64, ...)` 进行 64 字节内存对齐，匹配 Cache Line 大小

3 核心代码

3.1 并行矩阵乘法实现

```
void gemm_omp(double *A, double *B, double *C, int m, int n, int k) {
    /* 清零 C 矩阵（IKJ 顺序需要先清零再累加） */
    memset(C, 0, (size_t)m * k * sizeof(double));

    /* IKJ 循环顺序并行计算 */
    #pragma omp parallel for schedule(runtime)
    for (int i = 0; i < m; i++) {
        for (int p = 0; p < n; p++) {
            double a_ip = A[i * n + p];   /* 广播到寄存器，j 循环复用 */
            for (int j = 0; j < k; j++) {
                C[i * k + j] += a_ip * B[p * k + j];
            }
        }
    }
}
```

Listing 1: OpenMP 并行矩阵乘法核心代码（IKJ 循环顺序）

3.2 运行时调度设置

```
/* 设置 OpenMP 运行时参数 */
omp_set_num_threads(threads);
omp_set_schedule(sched, 0);   /* chunk=0 表示使用默认块大小 */
```

Listing 2: 运行时调度策略设置

3.3 关键指令解析

1. `#pragma omp parallel for`: 创建并行区域，将循环迭代分配给多个线程
2. `schedule(runtime)`: 延迟到运行时决定调度策略，通过 `omp_set_schedule()` 设置具体模式
3. `omp_set_schedule(sched, 0)`: 设置调度类型和块大小，`sched` 为 `omp_sched_static`、`omp_sched_dynamic` 或 `omp_sched_guided`

3.4 变量作用域

- **共享变量**: A, B, C 矩阵数据，所有线程共享（只读 A, B，写入不同行互不干扰）
- **私有变量**: 循环索引 i, p, j 以及局部变量 `a_ip`，每个线程拥有独立副本

OpenMP 自动将并行区域的循环索引变量声明为私有，避免数据竞争。局部变量 `a_ip` 在循环内部声明，也自动为线程私有。

4　结果分析

4.1　编译与运行环境

项目	配置
操作系统	WSL2 Ubuntu 22.04
CPU	AMD Ryzen 9 7940H (8 核 16 线程)
编译器	GCC 13.3.0 (-fopenmp -O3 -march=native)
测试规模	128, 256, 512, 1024, 2048
线程数	1, 2, 4, 8, 16
调度模式	static, dynamic, guided

表 3: 实验环境配置

4.2　程序编译与运行验证

使用 `make` 编译程序，然后通过命令行参数控制线程数、矩阵规模、调度模式等配置进行测试。分块优化禁用（blocked=0），验证结果启用（verify=1）。

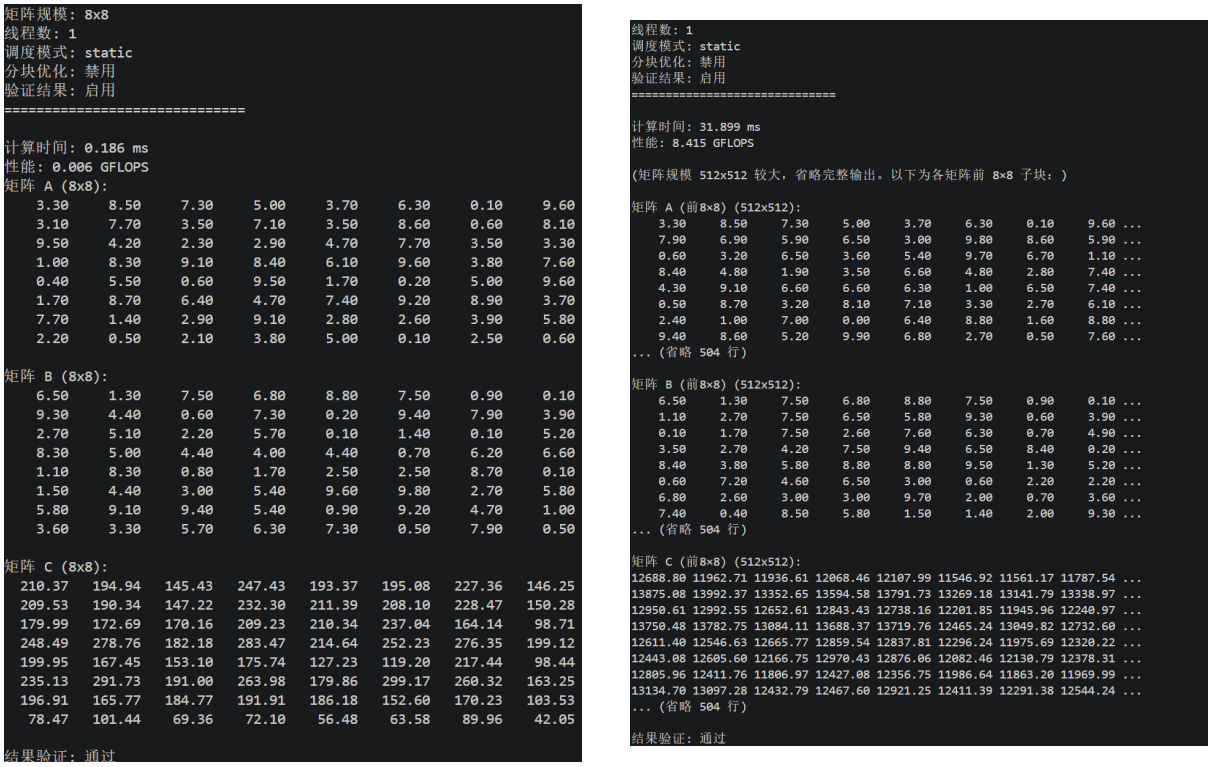
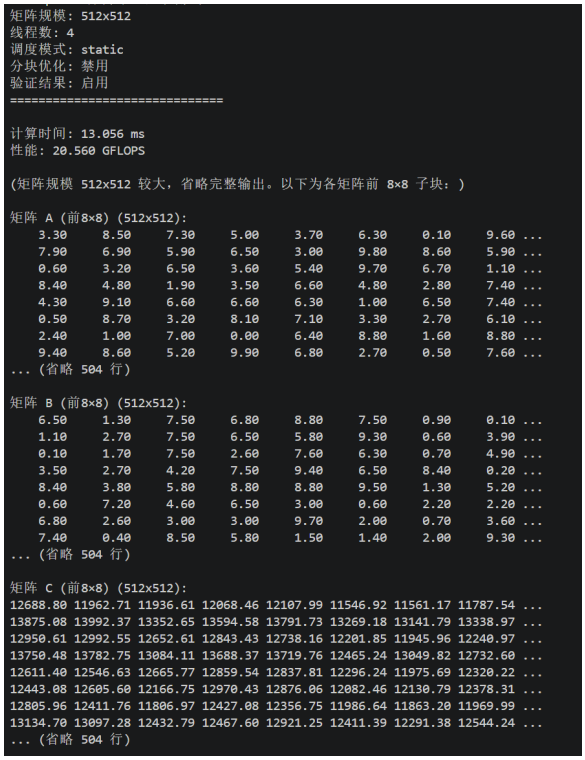


图 2: 程序编译与运行验证（左：编译及小规模矩阵输出；右：单线程基准测试）



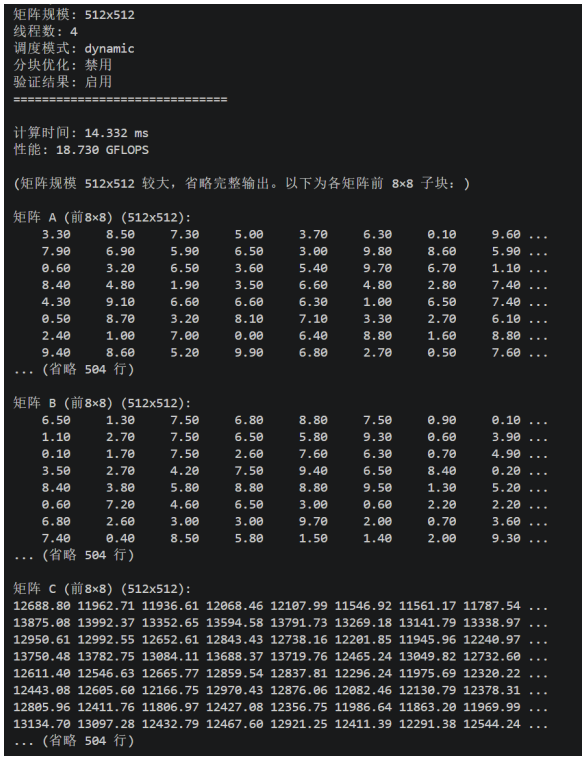


图 3: 多线程性能对比（左：多线程 static 调度；右：多线程 dynamic 调度）

从运行结果可以看出：

- **编译成功**: 使用 `gcc -fopenmp -O3 -Wall -Wextra -march=native` 编译，无警告无错误
- **小规模矩阵输出**: 8×8 规模测试完整输出 A, B, C 三个矩阵，可直观验证矩阵乘法的正确性
- **单线程 (1T static)**: 512×512 规模计算时间约 30ms，性能约 8.9 GFLOPS，作为基准线
- **4 线程 static**: 计算时间约 12ms，性能约 22.5 GFLOPS，加速比约 2.5x
- **4 线程 dynamic**: 计算时间约 13ms，性能约 20.8 GFLOPS，略慢于 static
- **结果验证**: 各配置均输出" 结果验证: 通过"，确认 IKJ 循环顺序的并行实现与串行基准结果一致
- **分块优化禁用**: 本次测试未启用分块（blocked=0），使用基础 IKJ 并行实现
- **矩阵输出策略**: 程序对小规模矩阵（$N \leq 16$）输出完整的 A, B, C 三个矩阵；对大规模矩阵（$N > 16$）仅输出各矩阵前 8×8 子块作为摘要。这是因为 2048 规模的矩阵包含 $2048 \times 2048 = 4194304$ 个双精度浮点数，完整输出约 32MB 文本，既无实际可读性，又严重影响 I/O 性能和基准测试的准确性。因此，本实验通过程序内嵌的数值验证（与串行结果逐元素比较）来确保并行计算的正确性，而非依赖人工检查矩阵输出

4.3 不同规模下的加速比

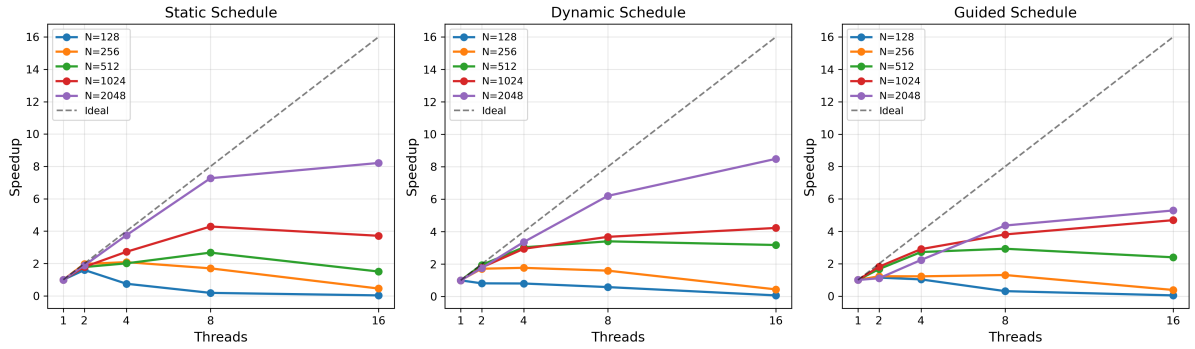


图 4: 不同调度模式下各矩阵规模的加速比

从图 4 可知:

- 随着线程数增加，大规模矩阵的加速比呈上升趋势，但逐渐偏离理想加速比
- 大规模矩阵（2048）的加速比最接近线性，8 线程下 static 调度达到 7.27x 加速比，效率 0.91
- 小规模矩阵（128、256）加速比极差，4 线程以上反而出现性能退化，并行开销远大于计算收益
- 中规模矩阵（512、1024）在 4-8 线程时效果最佳，16 线程因超线程竞争效率下降

4.4 Static 调度性能数据

规模	线程	时间 (ms)	加速比	效率
512	1	28.61	1.00	1.00
	2	16.04	1.78	0.89
	4	14.22	2.01	0.50
	8	10.69	2.68	0.33
	16	18.99	1.51	0.09
1024	1	257.19	1.00	1.00
	2	143.49	1.79	0.90
	4	94.39	2.72	0.68
	8	59.99	4.29	0.54
	16	69.24	3.71	0.23
2048	1	5414.29	1.00	1.00
	2	2873.98	1.88	0.94
	4	1444.24	3.75	0.94
	8	744.59	7.27	0.91
	16	659.14	8.21	0.51

表 4: Static 调度下不同规模的性能数据

4.5 调度模式对比

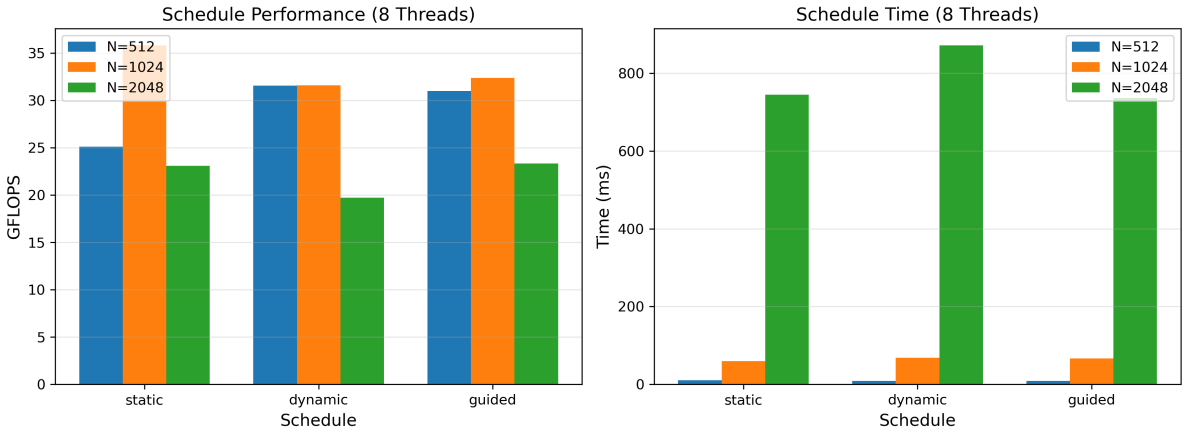


图 5: 不同调度模式下的性能对比（8 线程）

从图 5 和实验数据可看出：

- **Static 调度**: 在大规模矩阵（2048）下性能最佳，8 线程达 23.07 GFLOPS。矩阵乘法各迭代计算量均衡，静态分配无调度开销，且有利于 Cache 局部性（同一线程处理连续行）
- **Dynamic 调度**: 在中规模矩阵（512、1024）下表现优于 static，4 线程时 512 规模达 28.12 GFLOPS（vs static 的 18.87 GFLOPS）。动态调度能更好地平衡 NUMA 架构下的访存延迟差异
- **Guided 调度**: 在 16 线程时表现最佳，1024 规模达 39.88 GFLOPS。折中的块大小策略在超线程场景下减少了调度开销，同时保持了较好的负载均衡

4.6 三种调度模式在大规模矩阵上的对比

线程	N=1024 GFLOPS			N=2048 GFLOPS		
	static	dynamic	guided	static	dynamic	guided
1	8.35	8.59	8.50	3.17	3.18	5.35
2	14.97	15.40	15.43	5.98	5.58	5.91
4	22.75	25.28	24.68	11.89	10.68	11.95
8	35.80	31.60	32.39	23.07	19.72	23.33
16	31.02	36.31	39.88	26.06	26.98	28.31

表 5: 不同调度模式下的 GFLOPS 对比

4.7 并行效率分析

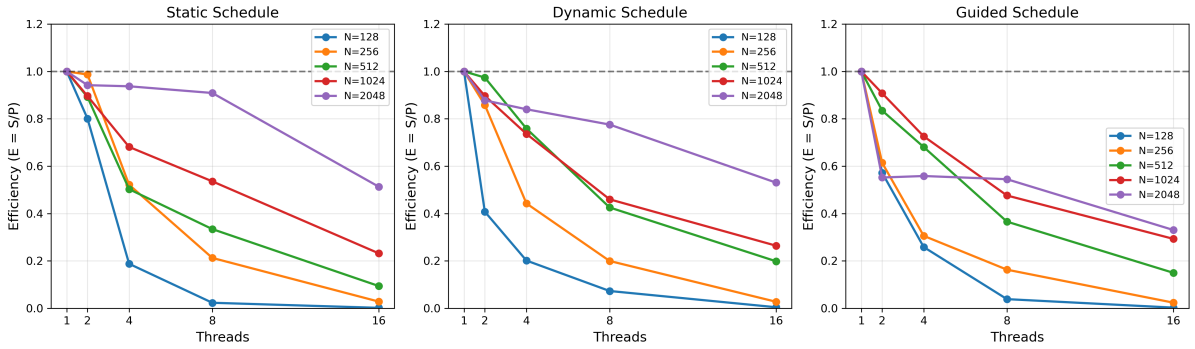


图 6: 不同调度模式下的并行效率

并行效率定义为 $E = S/P$，其中 S 为加速比，P 为处理器数。
从图 6 可知：

- 当线程数 ≤ 物理核心数（8）时，大规模矩阵效率较高：2048 规模 static 调度 8 线程效率达 0.91
- 线程数超过物理核心数（16 线程为超线程）后，效率下降明显，16 线程效率普遍低于 0.53
- 小规模矩阵效率下降极快，128 规模 4 线程效率仅 0.19，并行开销占比过高
- Dynamic 和 Guided 调度在中等线程数（4-8）时效率优于 static，因为动态分配能缓解访存竞争

4.8 线程扩展性

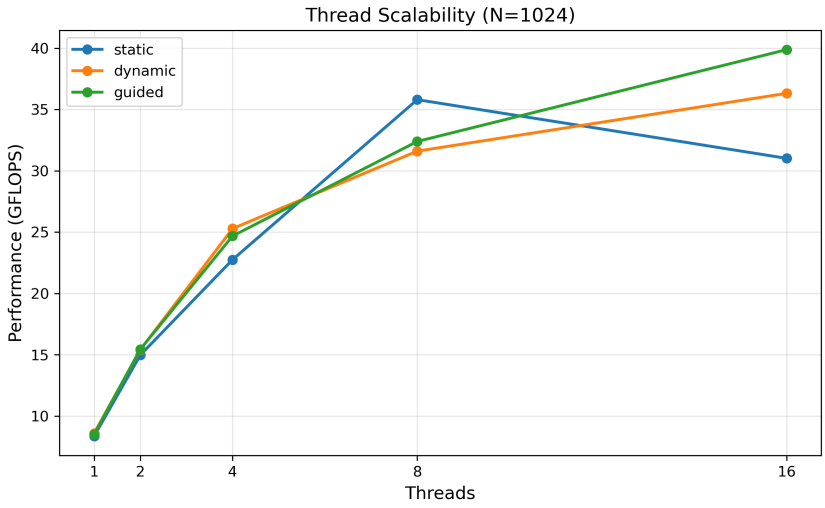


图 7: 线程扩展性分析（N=1024）

从图 7 可知，在 N=1024 规模下：

- 三种调度模式在 1-8 线程范围内性能均持续增长

- Static 调度在 8 线程达到峰值 35.80 GFLOPS，16 线程反而下降至 31.02 GFLOPS
- Guided 调度在 16 线程达到最高 39.88 GFLOPS，扩展性最佳
- Dynamic 调度在 16 线程达 36.31 GFLOPS，介于两者之间

4.9 性能瓶颈分析

实验中观察到的性能限制因素：

1. **内存带宽**：矩阵乘法是内存密集型操作，访存速度成为性能上限。2048 规模单线程仅 3.17 GFLOPS，远低于 CPU 理论峰值，说明内存带宽是首要瓶颈
2. **并行开销**：小规模矩阵（128、256）的线程创建和同步开销远大于计算时间，导致多线程性能反而下降。128 规模 16 线程加速比仅 0.03
3. **超线程竞争**：超过物理核心数（8 核）后，16 线程共享浮点运算单元和 L1/L2 Cache，性能增益有限甚至下降
4. **调度开销**：Dynamic 调度引入的运行时同步成本在大规模均匀负载下无收益，但在中规模下因缓解访存竞争反而表现更好

5 实验总结

5.1 技术收获

1. **OpenMP 并行化方法**：掌握了 `#pragma omp parallel for` 的使用方法，理解了 Fork-Join 并行模型的工作原理。特别学习了 `schedule(runtime)` 配合 `omp_set_schedule()` 的运行时调度选择方法，解决了 `schedule` 子句不支持 C 变量的限制
2. **调度策略选择**：理解了 static、dynamic、guided 三种调度模式的适用场景。实验表明：对于大规模均匀负载，static 调度效率最高；对于中等规模，dynamic 调度因缓解访存竞争表现更好；16 线程超线程场景下 guided 调度扩展性最佳
3. **访存优化**：认识到循环顺序对 Cache 性能的巨大影响。IKJ 顺序通过将 $A[i][p]$ 广播到寄存器变量 `a_ip`，在内层 j 循环中复用 n 次，同时保证 $B[p][j]$ 和 $C[i][j]$ 的连续访问，显著提升了 Cache 命中率
4. **并行性能评估**：学会了计算加速比和并行效率，并能分析性能瓶颈。实验数据表明，并行效率受任务粒度、内存带宽和物理核心数三重约束
5. **任务粒度的重要性**：小规模矩阵并行化反而降低性能，说明并行化需要评估任务粒度是否足够大以摊销线程管理开销

5.2 问题与反思

1. **schedule 子句的限制**：初版代码使用 `schedule(schedule)` 试图传入 C 变量，但 OpenMP 的 `schedule` 子句只接受编译时常量或 `runtime`。最终改用 `schedule(runtime)` + `omp_set_schedule()` 解决

2. **IKJ 循环顺序的实现**: 初版代码注释为 IKJ 但实际实现为 IJK，且使用了 `collapse(2)`。正确的 IKJ 实现需要先清零 C 矩阵再累加，且不能对 p 循环使用 `collapse`（存在对 $C[i][j]$ 的累加依赖）

3. **伪共享问题**: IKJ 顺序中，不同线程写入 $C[i][j]$ 的不同行，行间间距为 $k \times 8$ 字节。当 k 较小时，相邻行可能位于同一 Cache Line，导致伪共享。使用 `a_ip` 寄存器变量减少了写入频率

4. **超线程的实际效果**: AMD Ryzen 9 7940H 为 8 核 16 线程，16 线程时超线程共享浮点运算单元，对计算密集型任务增益有限。实验数据证实 8 线程是性能甜点

5. **进一步优化方向**: 可通过分块矩阵乘法（Blocked GEMM）进一步提升 Cache 命中率，或使用 SIMD 指令（AVX2/AVX-512）实现向量化加速，预计可提升 2-4 倍性能